

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Практическая работа №4

Выполнила:

Красноперова Е. Д.
К3443

Проверил:

Добряков Д. И.

Санкт-Петербург,
2026 г.

Задача

Разделить монолитное бэкенд-приложение на микросервисы с использованием концепции database-per-service.

Спроектировать микросервисную архитектуру, определить границы сервисов и способы их взаимодействия.

Разделить исходную базу данных на отдельные базы данных, принадлежащие соответствующим сервисам.

Спроектировать необходимые для межсервисного взаимодействия HTTP API и описать их в OpenAPI.

Описать варианты запросов, ответов и возможных ошибок.

Сформировать последовательность действий для перехода от монолитного приложения из ЛР1 к микросервисной архитектуре.

1. Исходное приложение

Исходным состоянием является монолитное приложение из ЛР1 на Express, TypeScript, TypeORM и PostgreSQL. В монолите единая база данных содержала User, Recipe, Step, Ingredient, RecipeIngredient, Comment, Like, SavedRecipe и Subscription.

Функциональность исходного приложения включает регистрацию и вход, JWT-аутентификацию, профиль пользователя, публикацию и получение рецептов, поиск и фильтрацию рецептов, сортировку и пагинацию, просмотр страницы рецепта с фотографией, видео, шагами и ингредиентами, а также лайки, сохранение рецептов, комментарии и подписки.

2. Разделение на микросервисы

UserService отвечает за пользователей: регистрацию, вход, JWT-аутентификацию, профиль и публичный профиль.

RecipeService отвечает за рецепты и их внутреннюю структуру: Recipe, Step, Ingredient и RecipeIngredient. Сервис также отвечает за получение рецептов, поиск, фильтрацию, сортировку и пагинацию.

SocialService отвечает за социальные связи: Like, Comment, SavedRecipe и Subscription.

API Gateway является единой публичной точкой входа. Gateway не владеет бизнес-данными и маршрутизирует клиентские запросы к соответствующему микросервису.

3. Микросервисная архитектура и взаимодействие

Сервисы взаимодействуют синхронно по HTTP REST с JSON. Внутренние endpoints имеют префикс /internal и защищены заголовком X-Service-Token.

Пользовательский JWT используется для клиентских запросов. Каждый бизнес-сервис самостоятельно проверяет JWT там, где требуется авторизация. Gateway при необходимости получает user_id из проверенного JWT и передаёт пользовательский запрос дальше.

Межсервисные связи не реализуются через ORM-связи между разными базами. Например, SocialService хранит user_id и recipe_id как идентификаторы и перед выполнением операций проверяет связанные объекты через UserService и RecipeService.

RecipeService получает краткие сведения об авторе через UserService. Для списка рецептов RecipeService запрашивает пакетные социальные сводки у SocialService, чтобы сохранить в публичном контракте LP1 likes_count и возможность сортировки по количеству лайков.

UserService для разделов «мои рецепты» и «сохранённые рецепты» использует внутренние API владельцев соответствующих данных: RecipeService и SocialService. Для получения данных сохранённых рецептов UserService сначала получает recipe_ids из SocialService, затем запрашивает рецепты пакетно у RecipeService.

4. Database-per-service

UserService владеет базой recipes_users и таблицей users.

RecipeService владеет базой recipes_recipes и таблицами recipes, steps, ingredients, recipe_ingredients.

SocialService владеет базой recipes_social и таблицами likes, comments, saved_recipes, subscriptions.

Физических внешних ключей между базами нет. Внутри Recipe DB связи Recipe–Step и Recipe–RecipeIngredient сохраняются как обычные ORM-связи. В Social DB user_id и recipe_id являются идентификаторами объектов других сервисов, но не ссылками на таблицы чужой базы.

5. Внутренние API

UserService: GET /internal/users/{user_id} возвращает краткие данные пользователя; GET /internal/users/{user_id}/exists проверяет существование пользователя.

RecipeService: GET /internal/recipes/{recipe_id} возвращает рецепт для межсервисного использования; GET /internal/recipes/{recipe_id}/exists проверяет существование рецепта и возвращает author_id; GET /internal/recipes/by-author/{user_id} возвращает рецепты пользователя; POST /internal/recipes/by-ids выполняет пакетное получение рецептов.

SocialService: GET /internal/recipes/{recipe_id}/summary возвращает likes_count, comments_count и персональные признаки; POST /internal/recipes/summaries выполняет пакетное получение социальных сводок; GET /internal/users/{user_id}/saved возвращает идентификаторы сохранённых рецептов.

6. Запросы, ответы и ошибки

Для внутренних запросов используется X-Service-Token. При отсутствии или неверном токене сервис возвращает 401 UNAUTHORIZED.

Если пользователь или рецепт отсутствует, соответствующий сервис возвращает 404 USER_NOT_FOUND или 404 RECIPE_NOT_FOUND.

Ошибки авторизации пользователя и попытки изменения чужого ресурса возвращают 401 UNAUTHORIZED или 403 FORBIDDEN соответственно.

Конфликт социальных операций, например повторное добавление лайка или сохранения, возвращает 409.

Некорректные входные данные возвращают 422 `VALIDATION_ERROR` либо 400 `VALIDATION_ERROR` в существующем контракте LP1; недоступность зависимого сервиса обрабатывается как 503 `SERVICE_UNAVAILABLE`.

Внутренние API используют единый объект ошибки вида `{"error": {"code": "...", "message": "..."} }`.

7. Переход от монолита к микросервисам

1. Зафиксировать публичный REST-контракт и функциональность LP1.
2. Выделить доменные границы `UserService`, `RecipeService` и `SocialService`.
3. Создать три отдельные PostgreSQL базы: `recipes_users`, `recipes_recipes` и `recipes_social`.
4. Перенести таблицы в базы владельцев и убрать прямой доступ сервисов к чужим таблицам.
5. Перенести `User` и авторизацию в `UserService`.
6. Перенести `Recipe`, `Step`, `Ingredient` и `RecipeIngredient` в `RecipeService` и сохранить поиск, фильтрацию, сортировку, пагинацию и полный просмотр рецепта.
7. Перенести `Like`, `Comment`, `SavedRecipe` и `Subscription` в `SocialService`.
8. Заменить междоменные ORM-связи внутренними HTTP-вызовами.
9. Добавить API Gateway как единую точку входа.
10. Проверить интеграционные сценарии: регистрация - вход - получение рецептов - фильтрация - просмотр рецепта - like/save/comment - подписка - получение моих и сохранённых рецептов.
11. После успешной проверки отключить старые монолитные обработчики.

Вывод

В результате спроектирована микросервисная архитектура из API Gateway, `UserService`, `RecipeService` и `SocialService`. Каждый бизнес-сервис владеет собственной базой данных, а междоменные зависимости реализуются через защищённые HTTP API.